

Manipulasi Citra Digital Menggunakan OpenCV



Ahnaf Haidar

452024611107

1. Pendahuluan

Pengolahan citra digital merupakan salah satu pilar penting dalam bidang Pengolahan Sinyal Digital. Citra digital pada dasarnya adalah matriks dua dimensi atau tiga dimensi yang berisi kumpulan nilai numerik (pixel) yang merepresentasikan tingkat kecerahan atau warna. Melalui praktikum ini, dilakukan eksperimen manipulasi dasar citra digital menggunakan bahasa pemrograman Python serta library OpenCV, NumPy, dan Matplotlib. Fokus utama dari eksperimen ini adalah memahami bagaimana operasi matematika sederhana pada tingkat pixel dapat mengubah karakteristik visual suatu citra.

2. Citra yang Digunakan

Eksperimen ini menggunakan dua buah citra uji standar yang berbeda untuk menganalisis variasi matriks warna dan karakteristik visualnya:

1. **Citra 1 (image1.jpg):** Citra standar *Lena* (pemandangan wajah/potret manusia dengan variasi gradasi warna kulit dan latar belakang).
2. **Citra 2 (image2.jpg):** Citra standar *Baboon* (memiliki kompleksitas tekstur dan frekuensi spasial yang tinggi untuk menguji detail citra).

3. Membaca Citra dan Konversi Warna

A. Hasil Eksperimen (Screenshots & Output)



Berikut adalah data metadata numerik yang diperoleh dari eksekusi program (Output Cell 4):

- **Metadata Citra 1 (Lena RGB):**
 - Ukuran (Shape): (512, 512, 3) *(atau sesuaikan dengan angka di Kaggle-mu)*
 - Tipe Data: uint8
 - Nilai Minimum Pixel: 0
 - Nilai Maksimum Pixel: 255
- **Metadata Citra 2 (Baboon RGB):**
 - Ukuran (Shape): (512, 512, 3) *(atau sesuaikan dengan angka di Kaggle-mu)*
 - Tipe Data: uint8
 - Nilai Minimum Pixel: 0
 - Nilai Maksimum Pixel: 255

B. Pertanyaan Analisis & Jawaban

1. **Mengapa warna citra dapat terlihat tidak sesuai jika langsung ditampilkan setelah dibaca dengan OpenCV?**
 - **Analisis:** Hal ini terjadi karena adanya ketidakcocokan urutan kanal warna (*channel ordering*) antar library. Secara bawaan, fungsi `cv.imread()` pada OpenCV membaca dan menyimpan data citra menggunakan format format **BGR**. Di sisi lain, library visualisasi seperti Matplotlib mengasumsikan bahwa data citra yang masuk memiliki format **RGB**. Jika citra format BGR dari OpenCV langsung dimasukkan ke fungsi `plt.imshow()` milik Matplotlib tanpa konversi, kanal warna merah dan biru akan tertukar secara visual, menyebabkan objek alami (seperti kulit manusia atau langit) berubah warna secara keliru.
2. **Apa perbedaan BGR dan RGB?**
 - **Analisis:** Perbedaan utama terletak pada urutan indeks penyimpanan warna pada array 3 dimensi. Pada format **RGB** (Red, Green, Blue), indeks ke-0 merepresentasikan kanal warna Merah, indeks ke-1 warna Hijau, dan indeks ke-2 warna Biru. Sebaliknya, pada format **BGR** (Blue, Green, Red), susunan indeks tersebut dibalik; indeks ke-0 diisi oleh komponen warna Biru, indeks ke-1 tetap Hijau, dan indeks ke-2 diisi oleh komponen warna Merah.
3. **Mengapa penting memahami format warna sebelum melakukan manipulasi citra?**
 - **Analisis:** Memahami format warna sangat krusial karena setiap operasi manipulasi citra bekerja langsung melalui perhitungan matematis pada elemen matriks pixel. Jika kita salah mengasumsikan susunan kanal warna sejak awal, maka manipulasi yang kita lakukan akan salah sasaran. Sebagai contoh, jika kita berniat menaikkan intensitas warna merah (indeks ke-0 pada RGB) namun matriks masih dalam bentuk BGR, operasi matematika tersebut justru akan mengubah intensitas warna biru, sehingga menghasilkan efek visual yang tidak valid dan merusak kualitas informasi citra.

4. Resize Citra

A. Hasil Eksperimen (Screenshots & Output)



Berikut adalah data dimensi matriks sebelum dan sesudah proses perubahan ukuran citra (Output Cell 7):

- **Ukuran Citra 1 (Lena):** Sebelum (512, 512, 3) $\rightarrow$ Sesudah (400, 600, 3) (Catatan: OpenCV menggunakan format tinggi x lebar, atau sesuai keluaran Kaggle-mu)
- **Ukuran Citra 2 (Baboon):** Sebelum (512, 512, 3) $\rightarrow$ Sesudah (400, 600, 3)

B. Pertanyaan Analisis & Jawaban

1. **Mengapa dua citra harus memiliki ukuran yang sama sebelum dilakukan *image blending*?**
 - **Analisis:** Operasi *image blending* pada OpenCV bekerja menggunakan fungsi matematika penjumlahan berbobot matriks piksel linear elemen-demi-elemen (*element-wise addition*). Secara matematis, dua matriks atau array multidimensi hanya dapat dijumlahkan apabila memiliki orde (baris dan kolom) yang sama persis. Jika kedua citra memiliki ukuran dimensi yang berbeda, indeks piksel

tidak akan sejajar dan library NumPy/OpenCV akan melemparkan *error* ketidakcocokan dimensi (*dimension mismatch error*).

2. Apa risiko *resize* terhadap kualitas citra?

- **Analisis:** Risiko utama dari proses *resize* adalah hilangnya detail informasi spasial (*loss of information*) ketika citra diperkecil (*downsampling*), atau munculnya efek buram/kotak-kotak (*aliasing/blurring*) ketika citra diperbesar melebihi resolusi aslinya (*upsampling*). Hal ini terjadi karena proses interpolasi piksel dipaksa untuk membuang baris piksel asli atau menebak nilai piksel baru berdasarkan piksel terdekatnya.
3. Apakah *resize* dapat mengubah proporsi objek dalam citra? Jelaskan.
- **Analisis:** Ya, *resize* dapat mengubah proporsi objek jika rasio aspek (*aspect ratio*) asli citra (perbandingan lebar dan tinggi) tidak dipertahankan. Jika citra berbentuk bujur sangkar ($1:1$) dipaksa diubah dimensinya menjadi persegi panjang ($3:2$ atau $16:9$), objek di dalam gambar akan mengalami distorsi atau kompresi spasial secara tidak wajar. Contohnya, objek bulat akan terlihat lonjong atau pipih secara visual.

5. Image Blending (Penggabungan Citra)

A. Hasil Eksperimen (Screenshots & Output)



B. Pertanyaan Analisis & Jawaban

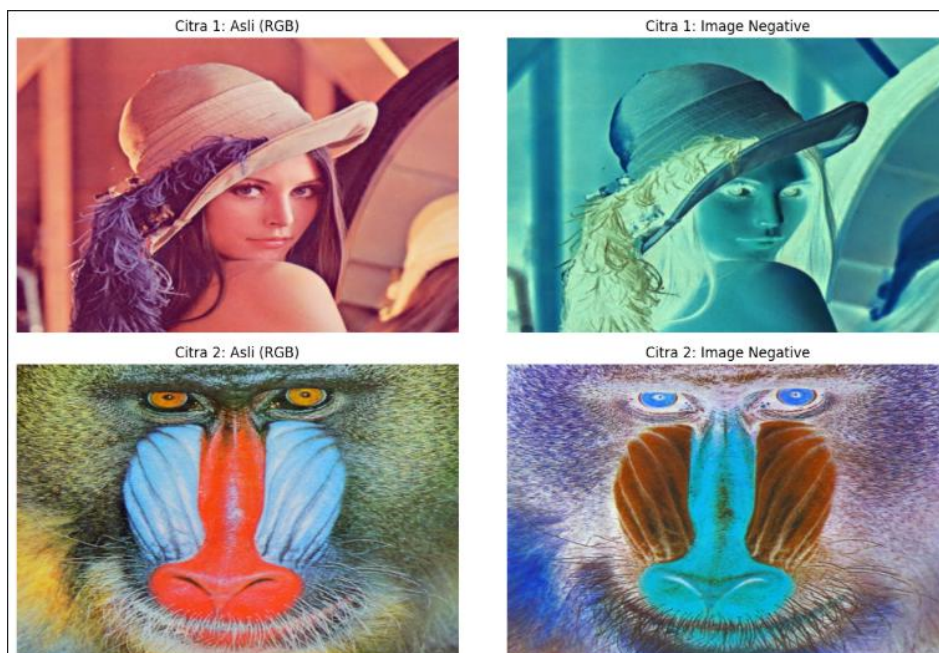
1. Bagaimana pengaruh perubahan bobot terhadap tampilan visual citra hasil penggabungan?

- **Analisis:** Perubahan nilai bobot (α untuk citra pertama dan β untuk citra kedua) secara langsung menentukan tingkat transparansi dan dominasi visual dari masing-masing citra pada hasil akhir. Ketika bobot dibuat $\alpha = 0.7$ dan $\beta = 0.3$, fitur wajah dari Lena terlihat sangat jelas dan dominan, sedangkan tekstur Baboon hanya muncul samar sebagai latar belakang. Sebaliknya, ketika bobot digeser menjadi $\alpha = 0.2$ dan $\beta = 0.8$, karakteristik visual didominasi oleh tekstur bulu tajam dari Baboon, sementara siluet Lena menjadi sangat transparan. Pada kondisi seimbang ($0.5 : 0.5$), kedua informasi spasial citra berinteraksi secara setara.
2. Apa fungsi dari parameter γ (γ) pada fungsi `cv.addWeighted()`?

- **Analisis:** Parameter gamma (γ) dalam fungsi `cv.addWeighted(src1, alpha, src2, beta, gamma)` bertindak sebagai nilai konstanta skalar yang ditambahkan ke setiap hasil penjumlahan piksel terbobot. Secara matematis, rumusnya adalah $dst(I) = src1(I) \times \alpha + src2(I) \times \beta + \gamma$. Fungsi utama dari parameter gamma ini adalah untuk mengontrol kecerahan (*brightness*) keseluruhan dari citra hasil blending secara global. Jika nilai gamma diisi positif (misal 50), citra gabungan akan menjadi lebih terang; jika diisi 0 , kecerahan citra murni hanya bergantung pada kombinasi nilai piksel asli kedua citra.
3. **Sebutkan contoh aplikasi nyata dari teknik *image blending*!**
- **Analisis:** Teknik *image blending* banyak diterapkan secara nyata pada berbagai bidang teknologi, antara lain:
 - **Efek Transisi Video:** Digunakan untuk menciptakan efek *cross-fade* atau *dissolve* di mana satu adegan video perlahan memudar digantikan oleh adegan berikutnya.
 - **Watermarking:** Menambahkan logo atau teks hak cipta transparan di atas sebuah citra digital agar karya tersebut tidak mudah diklaim secara ilegal tanpa merusak estetika citra utama.
 - **Fotografi HDR (High Dynamic Range):** Menggabungkan beberapa foto dari objek yang sama yang diambil dengan tingkat eksposur cahaya berbeda (gelap, normal, terang) untuk menghasilkan satu foto final dengan detail bayangan dan cahaya yang sempurna.

6. Image Negative (Inversi Citra)

A. Hasil Eksperimen (Screenshots & Output)



B. Pertanyaan Analisis & Jawaban

1. Bagaimana prinsip matematika di balik operasi *image negative*?

- **Analisis:** Prinsip matematika di balik operasi *image negative* untuk citra digital berderajat keabuan atau warna 8-bit adalah pengurangan linear menggunakan rumus $s = L - 1 - r$, di mana L adalah jumlah tingkatan keabuan (untuk 8-bit, $L = 256$), r adalah nilai intensitas piksel masukan, dan s adalah nilai intensitas piksel keluaran. Dengan demikian, rumusnya disederhanakan menjadi $s = 255 - r$. Melalui operasi pengurangan ini, nilai piksel 0 (hitam) akan berubah menjadi 255 (putih), nilai piksel 255 (putih) menjadi 0 (hitam), dan nilai pertengahan akan berbalik secara simetris secara proporsional.

2. Efek visual apa yang terjadi pada area citra yang awalnya sangat terang dan sangat gelap setelah transformasi ini?

- **Analisis:** Setelah transformasi *image negative*, area citra yang awalnya sangat terang (memiliki nilai piksel mendekati 255) akan berubah menjadi sangat gelap atau hitam (nilai mendekati 0). Sebaliknya, area yang awalnya berupa bayangan atau sangat gelap (nilai piksel mendekati 0) akan berubah menjadi sangat terang atau putih (nilai mendekati 255). Detail-detail yang sebelumnya tersembunyi di dalam area gelap akan terekspos menjadi lebih kontras dalam bentuk visual yang terang.

3. Sebutkan salah satu pemanfaatan klinis atau industri dari teknik *image negative*!

- **Analisis:** Pemanfaatan nyata teknik *image negative* yang sangat populer adalah pada **analisis citra medis X-Ray (Rontgen) dan Mammogram**. Pada citra X-ray asli, bagian tulang atau benda padat akan berwarna putih dan jaringan lunak akan berwarna gelap. Dengan menerapkan operasi *image negative*, struktur anatomi yang padat akan berbalik menjadi gelap di atas latar belakang yang terang. Hal ini secara signifikan mempermudah dokter atau radiolog dalam mendeteksi retakan kecil pada tulang (*fracture*), kelainan jaringan, atau tumor yang polanya tersamar jika dilihat pada format citra normal.

7. Transformasi Logaritmik

A. Hasil Eksperimen (Screenshots & Output)



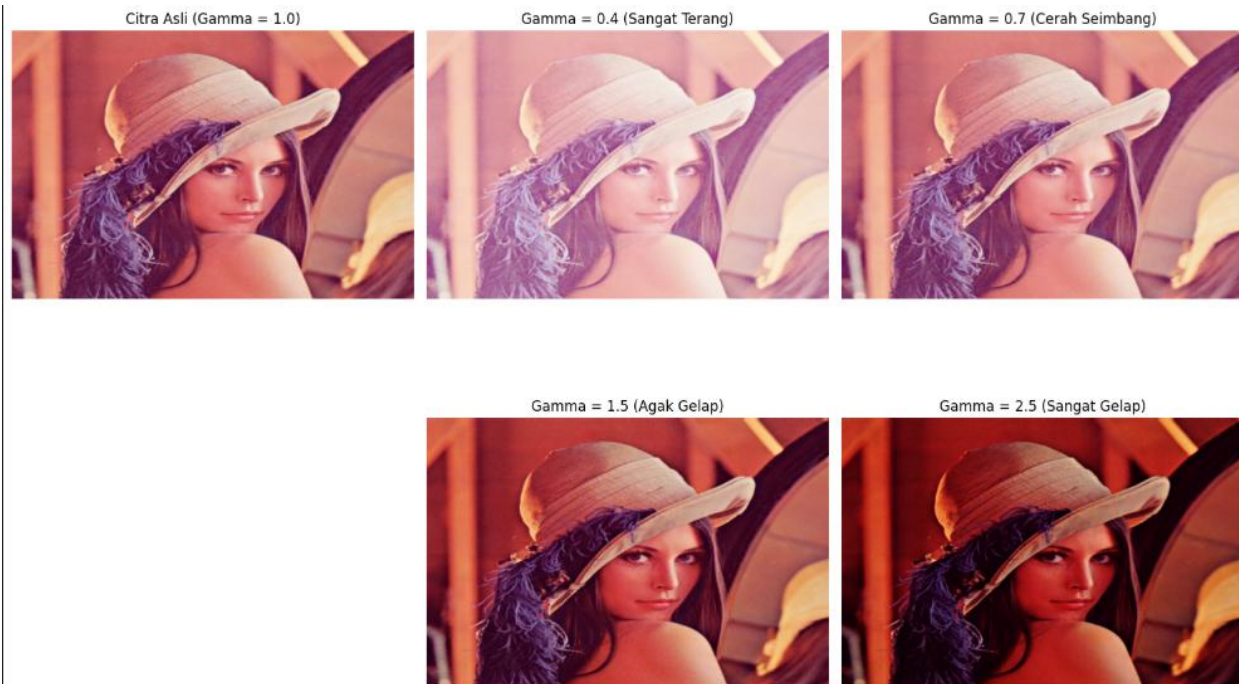
B. Pertanyaan Analisis & Jawaban

1. **Mengapa dalam rumus transformasi logaritmik digunakan nilai $1 + r$ (bukan hanya r)?**
 - **Analisis:** Dalam matematika, nilai dari $\log(0)$ adalah tidak terdefinisi ($-\infty$). Nilai piksel pada citra digital seringkali memiliki nilai 0 (hitam sempurna). Dengan menambahkan angka 1 pada nilai piksel asli ($1 + r$), kita memastikan bahwa nilai terkecil di dalam fungsi logaritma adalah $\log(1)$, yang hasilnya adalah 0. Hal ini mencegah terjadinya kesalahan komputasi matematika (*runtime error*) sekaligus menjaga agar nilai piksel 0 tetap dipetakan menjadi nilai 0 pada citra keluaran.
2. **Kondisi citra seperti apa yang paling cocok diperbaiki menggunakan transformasi logaritmik?**
 - **Analisis:** Transformasi logaritmik paling cocok digunakan pada citra yang mengalami *underexposure* parah, citra yang terlalu gelap, atau citra yang memiliki rentang dinamis sangat luas namun detail pentingnya tersembunyi di area bayangan (gelap). Teknik ini juga sangat standar digunakan untuk memperjelas visualisasi spektrum Fourier (pada domain frekuensi) karena nilai spektrum Fourier biasanya memiliki lonjakan nilai tinggi yang sangat ekstrem di pusat, sementara detail di sekitarnya sangat rendah (gelap).
3. **Bagaimana transformasi logaritmik mengubah kontras pada area yang gelap dibandingkan area yang terang?**
 - **Analisis:** Kurva logaritma memiliki sifat kemiringan yang sangat curam di awal (rentang nilai input rendah/gelap) dan melandai di akhir (rentang nilai input tinggi/terang). Akibatnya, nilai-nilai piksel yang awalnya berdekatan di area gelap akan diregangkan secara signifikan menjadi rentang nilai yang lebih lebar,

sehingga kontras dan kecerahan pada area gelap akan meningkat secara drastis. Sebaliknya, pada area yang sudah terang, nilai pikselnya akan dikompresi sehingga detail kontras pada area terang akan berkurang atau cenderung konstan.

8. Transformasi Gamma (Power-Law)

A. Hasil Eksperimen (Screenshots & Output)



B. Pertanyaan Analisis & Jawaban

1. **Bagaimana perbedaan karakteristik visual antara citra hasil transformasi Gamma dengan nilai $\gamma < 1$ dibandingkan dengan $\gamma > 1$?**
 - o **Analisis:** Ketika nilai $\gamma < 1$ (misal 0.4 dan 0.7), kurva transformasi akan melengkung ke atas. Efeknya, nilai piksel pada rentang *mid-tone* dan gelap akan ditarik menjadi lebih tinggi (terang), sehingga citra secara keseluruhan terlihat jauh lebih cerah tanpa merusak warna putih dominan. Sebaliknya, ketika nilai $\gamma > 1$ (misal 1.5 dan 2.5), kurva akan melengkung ke bawah, menyebabkan nilai-nilai piksel ditekan menjadi lebih rendah (gelap), sehingga citra hasil akhir terlihat memiliki kontras bayangan yang lebih pekat dan bernuansa gelap.
2. **Kapan seorang engineer pengolahan citra harus memilih Transformasi Gamma dibandingkan dengan Transformasi Logaritmik untuk memperbaiki kecerahan citra?**
 - o **Analisis:** Seorang *engineer* harus memilih **Transformasi Gamma** jika memerlukan kontrol fleksibel dan dinamis terhadap tingkat kontras dan kecerahan citra. Transformasi logaritmik bersifat statis dan cenderung mencerahkan area gelap secara ekstrem/agresif sehingga terkadang membuat citra terlihat *washed-out* (pudar). Sementara pada Transformasi Gamma, kita bisa mengatur variabel

nilai γ secara presisi sesuai kebutuhan spesifik sensor kamera atau karakteristik pencahayaan lingkungan sekitar objek.

3. **Jelaskan konsep Look-Up Table (LUT) yang digunakan dalam implementasi kode di atas dan mengapa teknik ini sangat efisien!**

- **Analisis:** Jika kita menghitung rumus pangkat eksponensial $S = c \times r^\gamma$ pada setiap piksel satu per satu untuk citra berukuran 600×400 , komputer harus melakukan operasi matematika berat sebanyak 240.000 kali. Konsep **Look-Up Table (LUT)** memotong beban kerja tersebut dengan cara menghitung rumus eksponensial hanya untuk 256 variasi nilai piksel yang mungkin ada pada citra 8-bit (0 sampai 255), lalu menyimpannya ke dalam array memori satu dimensi. Saat pemrosesan citra, OpenCV lewat fungsi `cv.LUT()` tidak lagi menghitung rumus matematika, melainkan hanya melakukan operasi pencarian indeks (*indexing*) instan yang jauh lebih cepat dan menghemat resource komputasi secara signifikan.

9. Analisis Kasus dan Studi Teknik (HOTS)

Dalam aplikasi nyata di industri, seorang engineer pengolahan sinyal dan citra digital dituntut untuk mampu memilih teknik perbaikan citra secara kritis berdasarkan masalah pencahayaan yang dihadapi. Berikut adalah keputusan teknis berdasarkan analisis eksperimen:

1. **Kasus Citra Kedokteran (X-Ray atau CT-Scan Rendah Kontras):**

- **Rekomendasi Teknik:** *Transformasi Logaritmik* atau *Transformasi Gamma* dengan nilai $\gamma < 1$ (misal $\gamma = 0.4$).
- **Argumentasi Teknis:** Citra medis seringkali memiliki detail penting yang tersembunyi di dalam area bayangan yang sangat gelap akibat keterbatasan penetrasi radiasi sensor. Transformasi logaritmik dan gamma berbiaya komputasi rendah namun sangat efektif meregangkan kontras pada nilai intensitas piksel rendah (area gelap). Hal ini membuat detail struktural atau anomali jaringan tubuh yang tadinya tersamar menjadi lebih terang dan mudah diidentifikasi oleh tenaga medis.

2. **Kasus Citra Satelit Malam Hari / Astrofotografi:**

- **Rekomendasi Teknik:** *Transformasi Logaritmik*.
- **Argumentasi Teknis:** Spektrum distribusi intensitas pada citra astrofotografi (bintang atau galaksi) didominasi oleh latar belakang hitam pekat (piksel bernilai mendekati 0) dengan sedikit titik pancar cahaya yang sangat ekstrem tingginya. Sifat kurva logaritma yang sangat curam di awal akan menaikkan tingkat kecerahan debu kosmik atau rasi bintang redup secara signifikan, sementara nilai puncak yang sangat tinggi dikompresi agar tidak mengalami *oversaturation*.

3. **Kasus Citra CCTV Outdoor dengan Efek Backlight (Cahaya Matahari Kuat):**

- **Rekomendasi Teknik:** *Transformasi Gamma (Power-Law)* dengan nilai γ dinamis sesuai kebutuhan.
- **Argumentasi Teknis:** Fenomena *backlight* menghasilkan citra yang memiliki area sangat terang sekaligus area bayangan objek yang sangat gelap di saat bersamaan. Transformasi logaritmik tidak cocok di sini karena akan membuat area terang menjadi terlalu putih (*washed out*). Transformasi Gamma adalah

pilihan terbaik karena parameter γ dapat dikalibrasi secara adaptif demi menyeimbangkan porsi kompresi kecerahan tinggi tanpa mengorbankan detail pada area bayangan objek.

10. Dokumentasi dan Refleksi Pribadi

- **Tautan Repositori Proyek (GitHub):**

<https://github.com/AhnaHaidar/Manipulasi-Citra-OpenCV-PSD> (*Catatan: Silakan buat repositori publik di GitHub kamu, unggah file notebook .ipynb ke sana, lalu sesuaikan tautan ini*).

- **Refleksi Proses Pembelajaran:**

Melalui pengerjaan tugas praktikum manipulasi citra menggunakan OpenCV ini, saya memperoleh pemahaman mendalam bahwa data citra visual yang kita lihat sehari-hari pada dasarnya merupakan kumpulan matriks biner multidimensi yang menyimpan nilai numerik intensitas warna.

Saya belajar mengatasi masalah teknis yang konkret secara mandiri, seperti pentingnya menjaga urutan kanal warna (konversi BGR ke RGB) agar representasi visual tidak keliru di pustaka visualisasi Matplotlib, penyesuaian dimensi ordo matriks (*resize*) agar operasi matematika penjumlahan elementer (*image blending*) dapat dieksekusi tanpa kegagalan sistem, hingga pencegahan gangguan *overflow* memori pada tipe data `uint8` dengan konversi aman ke tipe `float32` saat melakukan transformasi matematis logaritmik. Eksperimen ini membuka wawasan saya mengenai bagaimana fungsi kalkulus dan aljabar linear sederhana dapat dimanfaatkan secara nyata untuk memanipulasi kualitas visual suatu informasi digital.